Universidade do Minho
Departamento de Informática
Licenciatura em Ciência de Dados

Sistemas Conexionistas para Aprendizagem Automática
3º Ano, 1º Semestre
Ano letivo 2026/2027

Enunciado Prático nº 4
13 de outubro de 2026

Tema

Construção e treino de uma Rede Neuronal.

Enunciado

Pretende-se, com esta ficha, que seja realizado um conjunto de tarefas que permitam uma maior compreensão do *PyTorch*. Pretende-se também promover um espírito crítico e de investigação na implementação do treino de uma rede neuronal.

Tarefas

Esta ficha encontra-se dividida em duas partes distintas.

1. Na primeira parte desta ficha prática pretende-se que seja efetuado o treino de uma rede neuronal. Devem, para esse efeito, utilizar e completar o seguinte excerto de código (procurar pela *tag* ***TODO*** - 14 no total) e analisar os resultados obtidos. Devem também experimentar variar o número de camadas e neurónios da rede neuronal, analisando e comparando os resultados obtidos:

```
import torch
from torch import nn
from torch.utils.data import DataLoader
from torchvision import datasets
from torchvision.transforms import ToTensor
import matplotlib.pyplot as plt

#pytorch version being used
print(torch.__version__)
#is there a GPU available?
print(TODO)

#reproducibility
torch.manual_seed(42)

#load mnist training and test data (ToTensor scales the pixels to
[0, 1])
train_ds = datasets.MNIST('data', train=True, download=True,
transform=ToTensor())
test_ds = datasets.MNIST('data', train=False, download=True,
transform=ToTensor())
```

```
#data shape and cardinality
print('Train data shape', TODO)
print('Test data shape', TODO)

print('Number of training samples', TODO)
print('Number of testing samples', TODO)

#plotting some numbers!
for i in range(25):
    plt.subplot(5,5,i+1)    #Add a subplot as 5 x 5
    plt.xticks([])          #get rid of labels
    plt.yticks([])          #get rid of labels
    plt.imshow(test_ds.data[i], cmap="gray")
plt.show()

#the DataLoader delivers the data in batches
train_loader = DataLoader(train_ds, batch_size=32, shuffle=True)
test_loader = DataLoader(test_ds, batch_size=1000)

#building a three-layer sequential model (the output layer returns
logits)
model = nn.Sequential(
    nn.Flatten(),    #each 28x28 image becomes a list of 784 values
    nn.Linear(784, 64), nn.ReLU(),
    TODO,
    nn.Linear(32, 10)
)

#loss function and optimizer
loss_fn = TODO
optimizer = torch.optim.Adam(model.parameters(), lr=1e-3)

#training it
for epoch in range(5):
    model.train()
    for x, y in train_loader:
        logits = TODO
        loss = TODO
        optimizer.zero_grad()
        TODO
        TODO
    print('Epoch %d - loss: %.4f' %(epoch+1, loss.item()))

#evaluating it
model.eval()
correct = 0
with torch.no_grad():
    for x, y in test_loader:
        predicted = TODO
        correct += (predicted == y).sum().item()
test_acc = correct / len(test_ds)
print('\nTest accuracy:', test_acc)

#finally, generating predictions (probabilities from the output of
the last layer)
print('\nGenerating predictions for the first fifteen samples...')
x_test, y_test = next(iter(test_loader))
with torch.no_grad():
    predictions = torch.softmax(model(TODO), dim=1)
print('Predictions shape:', predictions.shape)
for i, prediction in enumerate(predictions):
    #torch.argmax returns the INDEX with the largest value of a
tensor
    predicted_value = torch.argmax(prediction)
    label = TODO
    print('Predicted a %d. Real value is %d.' %(predicted_value,
label))
```